

WEEK 12

TASK 1 - Reverse Linked List

```
class Solution {  
    public ListNode reverseList(ListNode head) {  
  
        ListNode prev = null;  
        ListNode current = head;  
  
        while (current != null) {  
            ListNode next = current.next;  
  
            current.next = prev;  
  
            prev = current;  
            current = next;  
        }  
  
        return prev;  
    }  
}
```

TASK 2 - Relative Ranks

```
import java.util.*;  
  
class Solution {  
    public String[] findRelativeRanks(int[] score) {  
        int n = score.length;
```

```

int[] sorted = score.clone();

Arrays.sort(sorted);

String[] answer = new String[n];

for (int i = 0; i < n; i++) {
    int rank = n - Arrays.binarySearch(sorted, score[i]);

    if (rank == 1) {
        answer[i] = "Gold Medal";
    } else if (rank == 2) {
        answer[i] = "Silver Medal";
    } else if (rank == 3) {
        answer[i] = "Bronze Medal";
    } else {
        answer[i] = String.valueOf(rank);
    }
}

return answer;
}
}

```

TASK 3 - Sort Array By Parity

```

class Solution {
    public int[] sortArrayByParity(int[] nums) {
        int left = 0;

```

```

int right = nums.length - 1;

while (left < right) {
    if (nums[left] % 2 == 1 && nums[right] % 2 == 0) {
        int temp = nums[left];
        nums[left] = nums[right];
        nums[right] = temp;
    }

    if (nums[left] % 2 == 0) {
        left++;
    }

    if (nums[right] % 2 == 1) {
        right--;
    }
}

return nums;
}
}

```

TASK 4 - Distribute Candies

```

import java.util.*;

class Solution {
    public int distributeCandies(int[] candyType) {
        HashSet<Integer> set = new HashSet<>();
    }
}

```

```

        for (int candy : candyType) {
            set.add(candy);
        }

        return Math.min(set.size(), candyType.length / 2);
    }
}

```

TASK 5 - Isomorphic Strings

```

import java.util.*;

class Solution {
    public boolean isIsomorphic(String s, String t) {
        HashMap<Character, Character> map = new HashMap<>();
        HashSet<Character> used = new HashSet<>();

        for (int i = 0; i < s.length(); i++) {
            char a = s.charAt(i);
            char b = t.charAt(i);

            if (map.containsKey(a)) {
                if (map.get(a) != b) {
                    return false;
                }
            } else {
                if (used.contains(b)) {
                    return false;
                }
                map.put(a, b);
                used.add(b);
            }
        }
    }
}

```

```

    }

    map.put(a, b);
    used.add(b);
}

}

return true;
}
}

```

TASK 6 - Find All Anagrams in a String

```

import java.util.*;

class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();

        if (p.length() > s.length()) {
            return result;
        }

        int[] countP = new int[26];
        int[] countS = new int[26];

        for (char c : p.toCharArray()) {
            countP[c - 'a']++;
        }
    }
}

```

```
int window = p.length();

for (int i = 0; i < s.length(); i++) {
    countS[s.charAt(i) - 'a']++;

    // Keep window size equal to p
    if (i >= window) {
        countS[s.charAt(i - window) - 'a']--;
    }

    // Check if current window is an anagram
    if (i >= window - 1 && Arrays.equals(countP, countS)) {
        result.add(i - window + 1);
    }
}

return result;
}
```